

Prasoon: Biological Weather Engine (Service A)

Tech Stack: FastAPI backend with PostgreSQL (time-series) and Redis (cache). Use Prophet and LSTM (TensorFlow/PyTorch) for forecasting, Scikit-learn for analytics. Integrate external APIs: OpenWeatherMap, Twitter API, Google Trends, News API.

Service Architecture: Implement the following REST endpoints under `/api/v1/biological-weather`:

- **Outbreak Predictions:** `GET /current` (current risk scores), `GET /forecast/{days}` (7/14/21-day forecasts), `GET /region/{region_id}` (region-specific forecasts), `GET /timeline` (time-series data).
- **Environmental Drivers:** `GET /weather/{region_id}` (temperature, humidity, rainfall), `GET /mobility/{region_id}` (population movement), `GET /breeding-zones` (mosquito breeding risk areas).
- **Social Signals:** `GET /sentiment` (Twitter/social sentiment on symptoms), `GET /trends` (Google Trends spikes for symptoms), `GET /news-triggers` (recent outbreak news).
- **Heatmap Data:** `GET /geojson` (map rendering data), `GET /hotspots` (top 10 critical regions).

ML Tasks:

- **Time-Series Forecasting:** Build disease outbreak prediction models (7–21 days) using Prophet and LSTM.
- **NLP Sentiment Analysis:** Use BERT to analyze social media posts about relevant symptoms or concerns.
- **Clustering:** Apply K-means to identify geographic outbreak clusters for mapping hotspots.
- **Correlation Analysis:** Compute correlations between weather patterns (from environmental data) and disease risk scores to feed into risk models.

Database Schema: Set up tables in PostgreSQL:

- **outbreak_predictions:** (id, region_id, disease_type, risk_score, predicted_cases, confidence_level, prediction_date, forecast_days, created_at).
- **environmental_data:** (id, region_id, temperature, humidity, rainfall, air_quality_index, date, created_at).
- **social_signals:** (id, region_id, platform, keyword, sentiment_score, mention_count, trending, date).

Notes for Frontend Integration: Frontend screens will call Service A's endpoints for outbreak and environmental data. For example, on the Global Dashboard (`/dashboard`), use `GET /biological-weather/outbreak-predictions/current` and `GET /weather/{region}` to populate the risk and weather map layers. The Region Drill-Down (`/region/{id}`) will call `GET /forecast/10` and `GET /environmental-drivers/weather/{id}` to show local predictions and weather charts. The mobile Public Health Cast (`/health-cast`) uses risk score and weather info from Service A (`/sentiment` or `/trends`) for local alerts. Ensure CORS enabled so frontend can fetch these data.

Pranav: Frontend Lead & Deep-Tier Supply Chain Engine (Service B)

Service B (Deep-Tier Supply Engine)

Tech Stack: FastAPI backend with Neo4j (graph DB for supply chain network) and PostgreSQL (metadata). Use Redis for caching. For data collection, employ Scrapy, BeautifulSoup, Selenium. Integrate APIs: global news, trade data, company reports.

Service Architecture: Build `/api/v1/supply-chain` endpoints:

- **Graph Data:** `GET /full-graph` (complete supply chain graph), `GET /disease/{disease_id}` (supply chain nodes for a disease), `GET /node/{node_id}` (details for a single node), `GET /path/{start}/{end}` (find path between two nodes), `GET /fragility-score` (overall supply chain fragility).
- **Disruptions:** `GET /current` (active disruptions), `GET /predicted` (future disruptions), `GET /by-region/{region_id}`, `POST /report` (manual report), `GET /history`.
- **Inventory:** `GET /global-status` (global inventory levels), `GET /region/{region_id}`, `GET /item/{item_id}`, `POST /update` (update inventory), `GET /shortage-forecast`.
- **Suppliers:** `GET /all` (list suppliers), `GET /alternatives/{node_id}`, `GET /reliability-score`, `GET /capacity`.
- **Routing:** `POST /optimize` (optimize supply routing), `GET /recommendations` (redistribution suggestions), `GET /logistics/{item_id}`.

Graph Schema (Neo4j): Model nodes for Disease, Treatment, ActiveIngredient, Factory, RawMaterial, Region, Supplier. Example relationships: `(:Disease)-[:TREATED_BY]->(:Treatment)`, `(:Treatment)-[:CONTAINS]->(:ActiveIngredient)`, etc. Store properties on relationships like quantity, lead_time, reliability.

Database Schema (PostgreSQL):

- **disruption_events:** (id, node_id, node_type, disruption_type, severity, start_date, end_date, description, source_url, verified, created_at).
- **inventory_snapshots:** (id, region_id, item_id, quantity, unit, timestamp, source, created_at).
- **suppliers:** (id, name, country, contact_info, reliability_score, capacity, specialization, certifications, updated_at).
- **scraping_log:** (id, url, source, keywords_found, relevance_score, processed, created_at).

Data Collection Pipeline: Schedule automated scraping every 6 hours from sources like Reuters, Bloomberg, government sites, and port statuses. Monitor keywords ("factory closure", "strike", "trade ban", etc). Use NLP to extract company names, locations, affected products, classify disruption type/severity, and link events to Neo4j nodes.

Frontend (React + Three.js) Responsibilities

Tech Stack: React.js with TypeScript, state via Redux Toolkit or Zustand. Use Leaflet/OpenStreetMap for 2D maps and Three.js (React-Three-Fiber) for the 3D globe. Charts with Recharts/D3.js. UI components via shadcn/ui and Tailwind CSS; animations with Framer Motion.

Screens & Integration: Connect all three services via the API Gateway (`/api/v1/biological-weather/*` → Service A, `/api/v1/supply-chain/*` → Service B, `/api/v1/nexus/*` → Service C). For each UI screen:

- **Screen 1: Global Dashboard** (`/dashboard`):

- **Data Integration:**

- Call **Service A:** `GET /biological-weather/outbreak-predictions/current` and `/forecast/{days}` for global outbreak risk layer; `/environmental-drivers/weather/{region}` and `/mobility/{region}` for temperature/humidity and movement layers.
 - Call **Service B:** `GET /supply-chain/fragility-score` or `/inventory/global-status` for the global supply stress layer; use `/shortage-forecast` to get regions with critical shortages to show as icons.
 - Call **Service C:** `GET /nexus/risk-analysis/critical-alerts` for the Critical Alerts feed; `GET /nexus/analytics/accuracy-metrics` and pattern detection for global stats if needed.

- **Components:** Render 3D globe with toggle layers: outbreak risk (red hotspots from Service A risk scores), environmental drivers, supply chain fragility (edges/arcs from Service B), mobility flows. Display shortage icons (🔗🔗) from Service B near cities.

- **Timeline Slider:** When the user moves or plays the timeline (0–21 days), fetch `GET /biological-weather/outbreak-predictions/forecast/{day}` dynamically to update map layers. The Play button toggles animation that increments the day.

- **Side Panel:**

- **Critical Alerts Feed:** Fetch from Service C's `/nexus/risk-analysis/critical-alerts` (output: alert type, region, severity). Auto-scroll through alerts.
 - **Global Statistics:** Compute from services: total active outbreaks (count from Service A), regions at risk (Service A count of regions with high risk), critical shortages count (from Service B's global inventory), supply chain disruptions (from Service B's `/disruptions/current`).
 - **Quick Action Buttons:**
 - *Run Simulation:* on click, POST scenario parameters to Service C's `/nexus/simulations/run` and switch to the Simulation screen.
 - *View Predictions:* may open a summary or call Service A's `/forecast/21` to display forecast overlay.
 - *Hospital Mode:* navigates to the Hospital Dashboard (`/hospital`).

- **Layer Toggle Panel:** Buttons to toggle each map layer; switching triggers React state changes to show/hide layers.

- **Screen 2: Region Drill-Down** (`/region/{region_id}`):

- **Data Integration:**

- **Service A:** GET `/biological-weather/outbreak-predictions/region/{region_id}` for local outbreak forecasts; `/environmental-drivers/weather/{region_id}` and `/breeding-zones` for temperature/humidity and breeding maps.
- **Service B:** GET `/supply-chain/inventory/region/{region_id}` for local inventory status; GET `/supply-chain/disruptions/by-region/{region_id}` for local supply issues.
- **Service C:** GET `/nexus/recommendations/hospitals?region={region_id}` for action recommendations (top items to stock, routing suggestions, etc).

- **Components:**

- **RegionHeader:** Show region name, alert score (0–100 from Service C risk calculation), risk level badge (e.g. “High”), last updated timestamp.
- **OutbreakPredictionsCard:** List diseases with next 10 days forecasts (use Service A data), show percentage change and graph, confidence. Also a 7-day trend chart.
- **InventoryStatusCard:** List critical items (from Service B inventory) with days remaining and mini bar chart. Show overall capacity gauge.
- **EnvironmentalFactorsCard:** Line/area charts for temperature, humidity, rainfall forecast (Service A data); include a mini map highlighting mosquito breeding zones (Service A).
- **LocalMapView:** Zoomed 2D map (Leaflet); show markers for nearby hospitals (static or from a hospital API), supply chain nodes (from Service B node data), and shortage icons (Service B shortage items).
- **ActionRecommendationsCard:** Display top 3 items to stock (from Service C output), supply routing suggestions (Service C), government notices (free text suggestions from Service C); include a **Download Report** button that triggers generation of a PDF or CSV of this region's data.
- **Back/Share Buttons:** Navigation controls.

- **Buttons/Actions:**

- Disease list items can be clicked to view details.
- *Download Report:* calls backend (maybe Service C or a report generator) to create and download region report.

- **Screen 3: Simulation Mode (`/simulation`):**

- **Data Integration:**

- **Service C:** Primary. On *Run Simulation*, POST selected scenario parameters to `/nexus/simulations/run`. Poll `GET /nexus/simulations/results/{simulation_id}` to retrieve results. Fetch pre-defined scenarios from `/nexus/simulations/scenarios`.

- **Components:**

- **SimulationControlPanel:** Dropdown for scenario selection (triggers pre-filled parameter sliders); sliders for rainfall change, mobility, infection multiplier, supply severity. *Run Simulation* button (triggers API call), *Reset* (resets parameters).
- **ComparisonView:** Split view showing Current State (left) vs Simulated State (right) maps and metrics. Overlay changes with arrows/% differences. Data for current state is from Service A/B (ongoing forecasts and supply); simulated state from Service C output.
- **ImpactSummaryCard:** Show predicted case changes, new hotspots, additional shortages, affected population, recommended actions (from simulation results).

- **TimelineComparison:** Dual-line charts (current vs simulated) for metrics over time. *Export Simulation Report* button to download data or PDF.

- **Buttons/Actions:**

- *Run Simulation:* Calls Service C to start Monte Carlo simulation (1000 runs, returns probability distributions).
- *Reset:* Clear simulation parameters and results.
- *Export Simulation Report:* Fetch result summary from Service C and download.

- **Screen 4: Hospital Collaboration Dashboard (/hospital):**

- **Data Integration:**

- **Service B:** GET /supply-chain/inventory/region/{hospital_region} for hospital inventory; POST /supply-chain/inventory/update when user uploads CSV or enters items.
- **Service C:** GET /nexus/risk-analysis/critical-alerts?region={hospital_region} for local outbreak alerts; GET /nexus/recommendations/hospitals?hospital_id={id} for recommended stockpiles and actions.
- **Service A:** Possibly use for hyper-local outbreak risk in hospital's area.

- **Components:**

- **HospitalProfileSection:** Display hospital name, location, preparedness score (0–100 from Service C), optional leaderboard rank, badges.
- **InventoryUploadCard:** Allow CSV/Excel upload or manual entry (item, qty, expiry). On upload/submission, POST to Service B to update. Show *Last Sync* timestamp from Service B.
- **ShortageAlertsCard:** List real-time shortage alerts (from Service B's GET /shortage-forecast or Service C). Each alert shows type, severity, estimated shortage date, action required tag. Include filter by severity.
- **CollaborationFeed:**
- *Nearby Hospital Status:* List nearby hospitals (could be stored or from API) with surplus items available (data from Service B if participating hospitals share inventory). Include *Contact* button (mailto or chat).
- *Request/Offer Marketplace:* Buttons “Request Item” and “Offer Surplus” open modals to create posts; list of active requests/offers (could call Service C or a custom endpoint for matchmaking).
- *Chat/Message System:* Real-time chat panel (could use WebSocket or Firebase) for hospital-to-hospital communication.
- **PredictiveInsightsCard:** Show incoming outbreak warnings (Service C alerts), recommended stockpile items (Service C recommendations), supply chain risk alerts (Service B disruptions), and a preparation timeline.
- **LeaderboardPanel (Optional):** If implemented, show top hospitals by preparedness score (data from Service C or internal DB), breakdown of criteria, and gamification badges.

- **Screen 5: Supply Chain Mapping View (/supply-chain):**

- **Data Integration:**

- **Service B:** Core data. `GET /supply-chain/graph-data/full-graph` to build network; `GET /supply-chain/node/{id}` to show node details; `GET /supply-chain/disruptions/current` and `GET /supply-chain/disruptions/history` for event timeline; `GET /supply-chain/suppliers/alternatives/{node_id}` for alternatives.
- **Service C:** Might also provide risk scores for nodes (via `/nexus/risk-analysis`).

- **Components:**

- **SupplyChainGraphView:** Render interactive network diagram with tiers (Disease at center, then Treatment, Active Ingredients, Factories, Raw Materials, Regions). Use nodes and edges data from Service B graph endpoints. Color or highlight edges/nodes based on fragility (from Service B fragility score) or disruptions.
- **Node Details Panel:** On hover/click a node, fetch and display details: current status, risk score (from Service C's `/impact-score`), connected dependencies, recent news (from Service B `scraping_log` or News API).
- **FilterPanel:** Controls to filter the graph by disease type, region, tier level, or show only critical paths (use Service B graph queries).
- **DisruptionTimelineCard:** List recent events: fetch from Service B `/disruptions/current` and `/disruptions/predicted`. Show event type (strike, flood, etc), affected node, impact, date. Also show upcoming predicted disruptions.
- **AlternativePathsFinder:** Let user select a problematic node (e.g., factory under disruption). On click *Find Alternative Suppliers*, call Service B `/suppliers/alternatives/{node_id}` to get a list. Show alternatives with supplier name, capacity, lead time, risk score. Allow user to select and *Export Recommendations* (export to CSV or update plan).
- **GlobalMapOverlay:** Overlay geographic data on a world map: factory location markers (Service B factories/regions), disruption icons (Service B disruptions), supply routes as animated arcs, and highlighted shortage regions (Service B `shortage-forecast`).

- **Screen 6: Analytics & Reports (`/analytics`):**

- **Data Integration:**

- **Service C:** Main source. `GET /nexus/analytics/accuracy-metrics` for prediction accuracy charts, `/nexus/analytics/pattern-detection` for discovered associations, `/nexus/analytics/accuracy-metrics` for specific metrics.
- **Service B:** Possibly for supply chain health score (`GET /supply-chain/fragility-score`).
- **Service A:** Historical vs actual outbreak data.

- **Components:**

- **DateRangePicker:** Set analysis period for all charts.
- **MetricsGrid:** Four summary cards showing: Total Predictions Accuracy (from C), Alerts Issued vs Actual (from C comparisons), Supply Disruptions Prevented (calculated from B/C logs), Regions Served (count).
- **PredictionAccuracyChart:** Line chart (predicted vs actual outbreaks by date), plus breakdown by disease type. Data from Service C's model performance logs.

- **SupplyChainHealthScore:** Gauge chart (0–100, from Service B's fragility score API), with trend line and contributing factors list (from Service B disruptions and inventory levels).
- **RegionalPerformanceHeatmap:** World/regional map with performance heatmap (e.g. by accuracy or response metrics); click a region to view details in a sidebar.
- **ExportSection:** Buttons to export data: PDF Report (bundle all visible data via backend service C or a report engine), CSV Data Export (raw metrics via Service C or B), and link to API docs (static page).
- **CustomReportBuilder:** UI to select metrics checkboxes, date range, region filters; on *Generate Report* click, call Service C or a reporting microservice to produce a custom PDF/CSV.

• **Screen 7: Settings & Configuration** (`/settings`):

• **Data Integration:** Mostly static, but may show API status via Service A/B endpoints.

• **Components:**

- **UserProfile:** Form to update name, email, role. Call User API (could be Service C or separate auth service) to update.
- **NotificationPreferences:** Toggles and sliders for alert preferences; save settings via backend (could be Service C or a user prefs service).
- **MapPreferences:** Controls for default zoom/center, animation speed, default layers; save in local storage or backend.
- **DataSources:** Display status of connected APIs (read endpoints like Service A's last sync times, etc). *Refresh Data* button to trigger data sync jobs (calls backend task).
- **AccessControl (admin only):** Manage users, roles, API keys (backed by user DB on Service C).
- **SystemInfo:** Show app version, TOS, Privacy (static content).

• **Screen 8: Public Health Cast (Mobile,** `/health-cast`):

• **Data Integration:**

- **Service A:** `GET /biological-weather/outbreak-predictions/current?region={user_region}` for local risk score; `GET /environmental-drivers/weather/{user_region}` for weather info.
- **Service C:** `GET /nexus/risk-analysis/critical-alerts?region={user_region}` for hyper-local warnings; `GET /nexus/recommendations/behavioral` (if created) or static tips for daily health. `GET /nexus/recommendations/hospitals?region={user_region}` for "Risk-Based Advice".
- **Service B:** `GET /supply-chain/inventory/region/{user_region}/pharmacies` for nearby pharmacy stock status (this may require a custom endpoint or integration with pharmacy inventory).

• **Components:**

- **LocationPermissionPrompt:** Ask user to allow location services.
- **UserLocationCard:** Show current city, overall risk score (Service A), current weather (Service A).
- **HyperLocalWarningsCard:** List of alerts from Service C (icon + type, severity, description, recommended action, with a "Learn More" button linking to resources).

- **NearbyPharmacyStatus:** Map view listing nearby pharmacies (use Google Places or local data) with stock status of critical items. *Call* and *Navigate* buttons for each (links to phone and maps). Filter by item type.
- **BehavioralNudgesCard:** Show daily health tip and risk-based advice (static or from Service C); include a **Share** button to let user share the tip.
- **UpcomingRisks:** 7-day forecast cards (date, risk type icon, severity, tip), populated from Service A's forecasts and Service C tips.
- **CommunityAlerts:** Section for user-reported local issues. Show list of alerts (with upvote/downvote). *Submit Alert* button opens a form (which could post to Service C or a new endpoint to broadcast to others).

Integration of Services: Use the API Gateway configuration to route requests. In frontend code, define API clients:

- **Service A Base URL:** `/api/v1/biological-weather`
- **Service B Base URL:** `/api/v1/supply-chain`
- **Service C Base URL:** `/api/v1/nexus`

Make sure to handle asynchronous data loading and caching (Redux or React Query). Use WebSockets or long-polling to subscribe to real-time alerts from Service C (published over RabbitMQ).

Buttons & Actions Summary:

- *Run Simulation* (Screen1): POST to Service C `/simulations/run`.
- *View Predictions* (Screen1): Navigate to a predictions UI or refresh outbreak layer (calls Service A).
- *Hospital Mode* (Screen1): Switch to `/hospital`.
- *Download Report* (Region view): Call Service C or dedicated report API to generate CSV/PDF.
- *Find Alternative Suppliers* (Screen5): Use Service B `/suppliers/alternatives`.
- *Export Recommendations* (Screen5): Trigger download of supplier recommendation list.
- *Export Simulation Report* (Screen3): Download data from Service C results.
- *PDF/CSV Export* (Screen6): Export data from Service C analytics.
- *Update Inventory* (Screen4): POST user input to Service B inventory update.
- *Submit Alert* (Screen8): POST new community alert (to Service C or DB).

Nithish: Nexus Intelligence Engine (Service C)

Tech Stack: FastAPI backend with PostgreSQL (analytics DB) and Redis (for real-time calculations). Use TensorFlow/PyTorch and Scikit-learn for ML, NetworkX for graph analysis. Use Celery + RabbitMQ for background jobs.

Service Architecture: Implement `/api/v1/nexus` endpoints:

- **Risk Analysis:** `GET /resilience-gap` (compute resilience gap), `GET /collision-points` (demand-supply collisions), `GET /critical-alerts` (high-priority alerts), `POST /calculate` (trigger new risk analysis run), `GET /timeline/{days}` (risk evolution time-series).
- **Recommendations:** `GET /hospitals/{hospital_id}` (hospital-specific action items), `GET /suppliers` (supplier opportunity recommendations), `GET /government` (policy suggestions), `GET /redistribution` (optimal redistribution plans).
- **Simulations:** `POST /run` (execute what-if simulation), `GET /results/{simulation_id}` (get

simulation output), `GET /scenarios` (list predefined scenarios), `DELETE /{simulation_id}` (delete simulation).

- **Predictions:** `GET /shortage-probability` (likelihood of item shortages), `GET /demand-spike` (predicted demand increases by region/item), `GET /supply-drop` (predicted supply decreases), `GET /impact-score` (combined impact metric per region/item).

- **Analytics:** `GET /accuracy-metrics` (model performance), `GET /pattern-detection` (emerging patterns from association rules), `POST /feedback` (ingest actual outcomes to retrain models).

ML Tasks:

1. Collision Detection:

- Consume outbreak forecasts from Service A and supply data from Service B every 30 minutes. Calculate predicted demand vs available supply for each region/disease. If `demand - supply > threshold`, trigger alert. Output risk scores (0-100), severity levels, and affected items/regions.

1. Recommendation Engine (Reinforcement Learning):

2. Train a policy-gradient model where state includes current inventory, forecasts, supply status; actions include redistributing stock, switching suppliers, stockpiling. Rewards minimize shortages and maximize preparedness. Continuously update with historical outcome data.

3. Pattern Detection (Clustering & Association Rules):

4. Use clustering to find correlated events (e.g. "Flood in region X → shortage in Y"). Apply Apriori on historical events and outcomes. Output patterns (e.g. "Holiday travel → flu spike"). Provide early warning rules (to be fed into alerts).

5. Impact Simulation (Monte Carlo):

6. For what-if scenarios, run 1000 random simulations varying parameters (rainfall, mobility, etc.) to generate probability distributions of outcomes. Provide confidence intervals for predicted cases, supply shortages, etc.

7. Optimization (Linear Programming):

8. Solve redistribution optimization: objective minimize shortages and cost. Constraints: supplier capacities, transportation lead times, demand requirements. Output optimal routing plans (fed to `/redistribution`).

Database Schema (PostgreSQL):

- **risk_calculations:** (id, region_id, item_id, predicted_demand, available_supply, gap, risk_score, alert_severity, calculation_date, forecast_date, created_at).

- **recommendations:** (id, target_entity_type, target_entity_id, recommendation_type, priority, action_items, expected_impact, status, created_at, expires_at).

- **simulation_results:** (id, simulation_id, scenario_type, parameters, predicted_outcomes, impact_summary, confidence_score, created_at).

- **model_performance:** (id, model_type, accuracy, precision, recall, mae, rmse, training_date, evaluation_date).

Integration with Frontend: Service C provides actionable data and models to the UI:

- **Global Dashboard:** `/nexus/risk-analysis/critical-alerts` populates the Critical Alerts Feed. Quick Action *Run Simulation* calls `/nexus/simulations/run`.
- **Region View:** `/nexus/recommendations/hospitals?region={}` fills the `ActionRecommendationsCard` with items to stock and suggestions.
- **Simulation Screen:** Handles simulation runs and results (as described above).
- **Hospital Dashboard:** `/nexus/risk-analysis/critical-alerts?region={hospital_region}` for local warnings; `/nexus/recommendations/hospitals?hospital_id={id}` for predictive insights (stockpile list, supply alerts).
- **Analytics:** Use `/nexus/accuracy-metrics` for the `PredictionAccuracyChart` and metrics cards; `/nexus/pattern-detection` for insights.
- **Mobile App:** `/nexus/risk-analysis/critical-alerts?region={user_region}` for hyper-local warnings; any “behavioral advice” could be served via `/nexus/recommendations`.

Background Jobs: Every 30 minutes, trigger Celery tasks: fetch outbreak forecasts (Service A) and supply statuses (Service B), run collision detection algorithm, store results in `risk_calculations`, and publish any new alerts via RabbitMQ. Frontend should subscribe (WebSocket) to display alerts in real time.

Success Metrics & Monitoring: Track model performance (store in `model_performance`), forecast accuracy, and system health. Provide APIs `/accuracy-metrics` for dashboards. Set up logging of all ML tasks (Celery logs). Ensure endpoints meet performance (<2s response) and data freshness (news <6 hours).
